PostgreSQL 15.19 (psql)
Module 0–26 Hands-On Lab

Institute Training & Placement Management System

Copy-paste-ready queries with easy explanations: What it is • Why we use it • Requirement • Query • How it works

Tested syntax target: PostgreSQL / psql 15.19

MODULE 0 — COMPLETE PSQL PROJECT SETUP

Purpose: Build the institute database, create related tables, insert sample data, and verify the lab.

Why we use it: Every later query needs the same consistent tables and sample records.

First open PostgreSQL terminal:

psql -U postgres

It will ask for your PostgreSQL password.

You should see something like:

postgres=#

Check existing databases:

```
\l
```

Create our institute database:

```
CREATE DATABASE institute_db;
```

Connect to it:

```
\c institute_db
```

You should see:

You are now connected to database "institute_db"

Check current database:

```
SELECT current_database();
```

Now create the tables.

```
CREATE TABLE students (
    student_id INT PRIMARY KEY,
    student_name VARCHAR(100) NOT NULL,
    city VARCHAR(50),
    qualification VARCHAR(50),
    experience_years INT,
    phone VARCHAR(15)
);
CREATE TABLE courses (
    course_id INT PRIMARY KEY,
    course_name VARCHAR(100) NOT NULL,
    category VARCHAR(50),
    duration_days INT,
    course_fee NUMERIC(10,2)
);
CREATE TABLE trainers (
    trainer_id INT PRIMARY KEY,
    trainer_name VARCHAR(100) NOT NULL,
    specialization VARCHAR(100),
    experience_years INT
);
```

```
CREATE TABLE enrollments (
    enrollment_id INT PRIMARY KEY,
    student_id INT REFERENCES students(student_id),
    course_id INT REFERENCES courses(course_id),
    trainer_id INT REFERENCES trainers(trainer_id),
    enrollment_date DATE,
    status VARCHAR(30)
);
CREATE TABLE payments (
    payment_id INT PRIMARY KEY,
    student_id INT REFERENCES students(student_id),
    course_id INT REFERENCES courses(course_id),
    amount_paid NUMERIC(10,2),
    payment_date DATE,
    payment_status VARCHAR(30)
);
CREATE TABLE placements (
    placement_id INT PRIMARY KEY,
    student_id INT REFERENCES students(student_id),
    company_name VARCHAR(100),
    job_role VARCHAR(100),
    salary NUMERIC(10,2),
    placement_date DATE
);
```

Check tables:

```
\dt
```

You should see:

students

courses

trainers

enrollments

payments

placements

To understand the structure of a table:

```
\d students
\d courses
```

Now insert data.

```
INSERT INTO students VALUES
(1, 'Arun Kumar', 'Chennai', 'B.E', 2, '9000000001'),
(2, 'Priya', 'Chennai', 'B.Tech', 1, '9000000002'),
(3, 'Rahul', 'Bangalore', 'B.E', 3, '9000000003'),
(4, 'Sneha', 'Hyderabad', 'B.Sc', 0, '9000000004'),
(5, 'Karthik', 'Chennai', 'MCA', 4, '9000000005'),
(6, 'Divya', 'Bangalore', 'B.Tech', 2, '9000000006'),
(7, 'Vijay', 'Chennai', 'B.E', 5, '9000000007'),
(8, 'Meena', 'Coimbatore', 'M.Sc', 1, '9000000008'),
(9, 'Suresh', 'Hyderabad', 'B.Tech', 3, '9000000009'),
(10, 'Anjali', 'Chennai', 'BCA', 0, '9000000010'),
(11, 'Ramesh', 'Madurai', 'B.E', 6, '9000000011'),
(12, 'Deepa', 'Chennai', 'MCA', 2, '9000000012');
```

```
INSERT INTO courses VALUES
(101, 'AWS Data Engineering', 'Data Engineering', 60, 30000),
(102, 'Azure Data Engineering', 'Data Engineering', 60, 32000),
(103, 'Generative AI', 'AI', 45, 35000),
(104, 'AWS DevOps', 'DevOps', 60, 28000),
(105, 'Azure DevOps', 'DevOps', 60, 28000),
(106, 'Python Developer', 'Development', 45, 20000),
(107, 'SQL Developer', 'Database', 30, 15000),
(108, 'PySpark Developer', 'Data Engineering', 45, 25000),
(109, 'Kubernetes Administrator', 'DevOps', 40, 22000);

INSERT INTO trainers VALUES
(201, 'Rajesh', 'AWS Data Engineering', 10),
(202, 'Kumar', 'Azure Data Engineering', 8),
(203, 'Sathish', 'Generative AI', 7),
(204, 'Manoj', 'DevOps', 12),
(205, 'Lakshmi', 'Python and SQL', 9),
(206, 'Ganesh', 'PySpark', 8);

INSERT INTO enrollments VALUES
(1001, 1, 101, 201, '2026-01-10', 'Active'),
(1002, 2, 102, 202, '2026-01-15', 'Active'),
(1003, 3, 103, 203, '2026-02-01', 'Completed'),
(1004, 4, 106, 205, '2026-02-05', 'Active'),
(1005, 5, 104, 204, '2026-02-10', 'Completed'),
(1006, 6, 108, 206, '2026-03-01', 'Active'),
(1007, 7, 105, 204, '2026-03-05', 'Completed'),
(1008, 8, 107, 205, '2026-03-10', 'Active'),
(1009, 9, 101, 201, '2026-04-01', 'Completed'),
(1010, 10, 103, 203, '2026-04-05', 'Active'),
(1011, 1, 108, 206, '2026-05-01', 'Active'),
(1012, 2, 106, 205, '2026-05-10', 'Completed');

INSERT INTO payments VALUES
(5001, 1, 101, 30000, '2026-01-10', 'Paid'),
(5002, 2, 102, 20000, '2026-01-15', 'Partial'),
(5003, 3, 103, 35000, '2026-02-01', 'Paid'),
(5004, 4, 106, 10000, '2026-02-05', 'Partial'),
(5005, 5, 104, 28000, '2026-02-10', 'Paid'),
(5006, 6, 108, 25000, '2026-03-01', 'Paid'),
(5007, 7, 105, 28000, '2026-03-05', 'Paid'),
(5008, 8, 107, 15000, '2026-03-10', 'Paid'),
(5009, 9, 101, 30000, '2026-04-01', 'Paid'),
(5010, 10, 103, 15000, '2026-04-05', 'Partial');

INSERT INTO placements VALUES
(7001, 3, 'TCS', 'GenAI Engineer', 800000, '2026-05-10'),
(7002, 5, 'Infosys', 'AWS DevOps Engineer', 750000, '2026-05-15'),
(7003, 7, 'Cognizant', 'Azure DevOps Engineer', 900000, '2026-06-01'),
(7004, 9, 'Accenture', 'AWS Data Engineer', 850000, '2026-06-10'),
(7005, 2, 'Wipro', 'Python Developer', 600000, '2026-06-15');
```

Verify the data:

```
SELECT * FROM students;
SELECT * FROM courses;
SELECT * FROM trainers;
SELECT * FROM enrollments;
SELECT * FROM payments;
SELECT * FROM placements;
```

Now your PSQL lab is ready.

MODULE 1 — SELECT

What is SELECT?

```
SELECT retrieves data from a table.
```

Why do we use it?

Data is already stored inside PostgreSQL. When business users, applications, reports, or developers need that data, we use SELECT.

For example, management says:

Show me all students.

```
SELECT *
FROM students;
```

How it works:

```
SELECT → What should I display?
*      → All columns
FROM   → From which table?
students → students table
```

If management needs only names and cities:

```
SELECT student_name, city
FROM students;
```

Why is this better than SELECT *?

Because if we only need two columns, there is no reason to retrieve every column.

MODULE 2 — DISTINCT

What is DISTINCT?

DISTINCT removes duplicate values from query output.

Why do we use it?

Our students table contains Chennai many times.

If management asks:

Which cities do our students come from?

We don't want:

Chennai

Chennai

Bangalore

Hyderabad

Chennai

Bangalore

Chennai

...

We want unique cities.

```
SELECT DISTINCT city
FROM students;
```

The existing project has students from five distinct cities.

To count them:

```
SELECT COUNT(DISTINCT city) AS total_cities
FROM students;
```

Think:

DISTINCT city

```
↓
Remove duplicate cities
↓
COUNT
↓
Count remaining unique cities
```

Result:

```
5
```

MODULE 3 — WHERE

What is WHERE?

```
WHERE filters rows.
```

Why do we use it?

Normally we don't always need the entire table.

Suppose HR asks:

Give me only Chennai students.

```
SELECT *
FROM students
WHERE city = 'Chennai';
```

How it works:

```
FROM students
    ↓
Take student records
WHERE city = 'Chennai'
    ↓
Keep only Chennai records
SELECT *
    ↓
Display them
```

For numeric data:

```
SELECT *
FROM students
WHERE experience_years > 2;
```

We can combine conditions:

```
SELECT *
FROM students
WHERE city = 'Chennai'
AND experience_years >= 2;
```

This means BOTH conditions must be true.

MODULE 4 — ORDER BY

What is ORDER BY?

ORDER BY sorts the result.

Why do we use it?

Suppose a student asks:

Show courses from cheapest to most expensive.

```
SELECT course_name, course_fee
FROM courses
ORDER BY course_fee ASC;
```

ASC means:

Ascending

```
Small → Big
A → Z
Old → New
```

For highest fee first:

```
SELECT course_name, course_fee
FROM courses
ORDER BY course_fee DESC;
```

DESC means descending:

```
Big → Small
Z → A
New → Old
The lab also demonstrates sorting by multiple columns.
```

MODULE 5 — LIMIT AND OFFSET

What are LIMIT and OFFSET?

```
LIMIT controls how many rows are returned.
OFFSET controls how many rows PostgreSQL skips first.
```

Why do we use them?

Mainly for pagination.

Imagine our website has:

1000 students

We don't want to display all 1000 on one page.

We want:

```
Page 1 → students 1-5
Page 2 → students 6-10
Page 3 → students 11-15
```

Page 1:

```
SELECT *
FROM students
ORDER BY student_id
LIMIT 5;
```

Page 2:

```
SELECT *
FROM students
ORDER BY student_id
LIMIT 5 OFFSET 5;
```

Meaning:

```
OFFSET 5 → Skip first 5
LIMIT 5 → Give next 5
```

Using ORDER BY here is important so pagination has a predictable order.

MODULE 6 — MIN AND MAX

What are MIN and MAX?

They are aggregate functions.

MIN() finds the smallest value.

MAX() finds the largest value.

Why do we use them?

Suppose management asks:

What is our cheapest course?

```
SELECT MIN(course_fee) AS lowest_fee
FROM courses;
```

Result:

```
15000
```

Then:

What is our most expensive course?

```
SELECT MAX(course_fee) AS highest_fee
FROM courses;
```

Result:

```
35000
```

Another business example:

```
SELECT MAX(salary) AS highest_salary
FROM placements;
```

This tells management the highest placement salary.

MODULE 7 — COUNT

What is COUNT?

COUNT() counts records.

Why do we use it?

This is extremely common in reporting.

Management may ask:

How many students do we have?

```
SELECT COUNT(*) AS total_students
FROM students;
```

Result:

```
12
```

Or:

How many students are from Chennai?

```
SELECT COUNT(*) AS chennai_students
FROM students
WHERE city = 'Chennai';
```

Here PostgreSQL first filters Chennai records and then counts them.

students

```
  ↓
WHERE city = Chennai
  ↓
COUNT
  ↓
6
```

MODULE 8 — SUM

What is SUM?

SUM() adds numeric values.

Why do we use it?

Very common for financial reports.

Management asks:

How much money has the institute collected?

```
SELECT SUM(amount_paid) AS total_collection
FROM payments;
```

The sample payment data produces a total of 236000.00.

If they ask:

How much did we collect only from fully paid students?

```
SELECT SUM(amount_paid) AS paid_collection
FROM payments
WHERE payment_status = 'Paid';
```

So:

```
WHERE
↓
Filters Paid records
SUM
↓
```

Adds their amount_paid

MODULE 9 — AVG

What is AVG?

AVG() calculates an average.

Why do we use it?

Suppose management asks:

What is our average course fee?

```
SELECT ROUND(AVG(course_fee), 2) AS average_course_fee
FROM courses;
```

AVG(course_fee) calculates the average.

ROUND(..., 2) displays it with two decimal places.

Another example:

```
SELECT ROUND(AVG(salary), 2) AS average_salary
FROM placements;
```

This answers:

What is the average placement salary?

MODULE 10 — LIKE AND ILIKE

What are LIKE and ILIKE?

They search text using patterns.

Why do we use them?

Suppose someone searches:

Give me all students whose names start with A.

```
SELECT *
FROM students
WHERE student_name LIKE 'A%';
```

% means any number of characters.

So:

'A%'

A + anything

Examples:

Arun

Anjali

Ajay

Ashok

For:

```
WHERE course_name LIKE '%Developer';
```

it means anything ending with Developer.

For:

```
WHERE course_name LIKE '%Data%';
```

it means Data can occur anywhere.

PostgreSQL also provides:

ILIKE

which performs case-insensitive matching. Your lab uses both forms.

MODULE 11 — IN / NOT IN

What is IN?

IN checks whether a value matches one of several values.

Why use it?

Suppose management wants:

Chennai, Bangalore and Hyderabad students.

Instead of:

```
WHERE city = 'Chennai'
OR city = 'Bangalore'
OR city = 'Hyderabad'
```

we can simply write:

```
SELECT *
FROM students
WHERE city IN ('Chennai', 'Bangalore', 'Hyderabad');
```

Much easier to read.

NOT IN does the opposite:

```
SELECT *
FROM students
WHERE city NOT IN ('Chennai', 'Bangalore');
```

We can also use a subquery:

```
SELECT *
FROM students
WHERE student_id IN (
   SELECT student_id
   FROM enrollments
);
```

Meaning:

Give me students whose IDs are present in the enrollment table.

MODULE 12 — BETWEEN

What is BETWEEN?

BETWEEN checks whether something falls inside a range.

Why use it?

Suppose a student says:

My budget is ₹20,000–₹30,000. Show suitable courses.

```
SELECT *
FROM courses
WHERE course_fee BETWEEN 20000 AND 30000;
```

Important:

BETWEEN is inclusive.

```
20000 → included
30000 → included
```

Dates can also be checked:

```
SELECT *
FROM enrollments
WHERE enrollment_date
```

BETWEEN DATE '2026-01-01'

```
    AND DATE '2026-03-31';
```

This returns enrollments in that date range.

MODULE 13 — ALIAS

What is an alias?

An alias gives a temporary readable name to a column or table.

Why use it?

Suppose our database column is:

student_name

but our report should show:

name

We write:

```
SELECT
    student_name AS name,
    city AS location
FROM students;
```

We also use table aliases:

```
SELECT
    s.student_id,
    s.student_name
FROM students AS s;
```

Now:

s = students

This becomes very important in JOIN queries because otherwise table names make queries very long.

MODULE 14 — BASIC JOIN

What is JOIN?

JOIN combines related information stored in different tables.

Why do we use it?

This is one of the most important SQL concepts.

Look at our database:

students

student_id = 1

student_name = Arun

enrollments

student_id = 1

course_id = 101

courses

course_id = 101

course_name = AWS Data Engineering

Management wants:

Arun Kumar | AWS Data Engineering

No single table contains both pieces of information.

So:

```
SELECT
    s.student_name,
    c.course_name,
    e.enrollment_date,
    e.status
FROM enrollments e
JOIN students s
    ON e.student_id = s.student_id
JOIN courses c
    ON e.course_id = c.course_id;
```

Your enrollment table is effectively the bridge connecting students to courses in this query.

MODULE 15 — INNER JOIN

What is INNER JOIN?

INNER JOIN returns only matching records.

Why use it?

Suppose we only care about students who actually enrolled.

```
SELECT
    s.student_id,
    s.student_name,
    c.course_name
FROM students s
INNER JOIN enrollments e
    ON s.student_id = e.student_id
INNER JOIN courses c
    ON e.course_id = c.course_id;
```

Think:

Student exists + Enrollment exists

```
          ↓
        SHOW
```

Student exists + No enrollment

```
          ↓
      DON'T SHOW
```

That's the key idea of INNER JOIN.

MODULE 16 — LEFT JOIN

What is LEFT JOIN?

```
LEFT JOIN keeps every record from the left table, even if there is no match on the right.
```

Why use it?

Suppose management asks:

Show every student. I also want students who haven't enrolled.

```
SELECT
    s.student_id,
    s.student_name,
    e.course_id
FROM students s
LEFT JOIN enrollments e
    ON s.student_id = e.student_id;
```

If no enrollment exists:

Ramesh | NULL

Deepa | NULL

Now we can find only non-enrolled students:

```
SELECT
    s.student_id,
    s.student_name
FROM students s
LEFT JOIN enrollments e
    ON s.student_id = e.student_id
WHERE e.student_id IS NULL;
```

Your sample data intentionally leaves Ramesh and Deepa without enrollment so this query produces a meaningful result.

This pattern is very important:

```
LEFT JOIN
```

+

IS NULL

=

Find missing records

MODULE 17 — RIGHT JOIN

What is RIGHT JOIN?

It keeps all records from the right table plus matching records from the left.

Why use it here?

We want every course, including courses with zero students.

```
SELECT
    e.enrollment_id,
    c.course_id,
    c.course_name
FROM enrollments e
RIGHT JOIN courses c
    ON e.course_id = c.course_id;
```

For Kubernetes Administrator, nobody enrolled.

Still:

NULL | 109 | Kubernetes Administrator

will appear.

That's because courses is on the right side.

MODULE 18 — FULL JOIN

What is FULL JOIN?

```
FULL JOIN keeps records from both sides.
```

It returns:

Matching records

+

Left records without matches

+

Right records without matches

Why use it?

Mainly when comparing datasets and you need to see both matched and unmatched information.

```
SELECT
    e.enrollment_id,
    e.student_id,
    c.course_id,
    c.course_name
FROM enrollments e
FULL JOIN courses c
    ON e.course_id = c.course_id;
```

Your lab uses it to compare enrollments against the full course list.

MODULE 19 — CROSS JOIN

What is CROSS JOIN?

```
CROSS JOIN creates every possible combination.
```

Why use it?

Suppose we want to generate every:

Student × Course

combination, maybe before building a recommendation or eligibility process.

```
SELECT
    s.student_name,
    c.course_name
FROM students s
CROSS JOIN courses c;
```

We have:

12 students

9 courses

12 × 9

=

```
108 records
Be careful with CROSS JOIN because large tables can create huge results.
```

MODULE 20 — UNION / UNION ALL

What is UNION?

```
UNION combines rows returned by separate SELECT statements.
```

This is different from JOIN.

```
JOIN
→ combines columns
UNION
→ combines rows
```

Suppose we want one list containing students and trainers:

```
SELECT student_name AS person_name
FROM students
UNION
SELECT trainer_name
FROM trainers;
```

Why use UNION?

Because both queries return the same kind of information: names.

```
UNION removes duplicates.
UNION ALL:
SELECT student_name AS person_name
FROM students
UNION ALL
SELECT trainer_name
FROM trainers;
```

keeps duplicates. This distinction is also captured in your lab.

MODULE 21 — GROUP BY

What is GROUP BY?

```
GROUP BY puts similar records into groups so we can calculate totals, counts, averages, etc.
```

Why use it?

Management doesn't always want individual records.

Sometimes they want a summary:

How many students are there in each city?

```
SELECT
   city,
```

COUNT(*) AS total_students

```
FROM students
GROUP BY city;
```

Think:

Arun Chennai

Priya Chennai

Karthik Chennai

Vijay Chennai

Anjali Chennai

Deepa Chennai

```
GROUP BY city
      ↓
Chennai becomes one group
      ↓
COUNT(*)
      ↓
6
```

So GROUP BY is heavily used with:

COUNT()

SUM()

AVG()

MIN()

MAX()

MODULE 22 — HAVING

What is HAVING?

```
HAVING filters results after grouping.
```

Why use it?

Suppose management asks:

Show only cities having more than one student.

First group:

```
SELECT
   city,
```

COUNT(*) AS total_students

```
FROM students
GROUP BY city;
```

Then filter those groups:

```
SELECT
   city,
```

COUNT(*) AS total_students

```
FROM students
GROUP BY city
HAVING COUNT(*) > 1;
```

The easiest way to remember:

```
WHERE
→ Filters rows
GROUP BY
→ Creates groups
HAVING
→ Filters groups
```

Your sample output leaves Chennai, Bangalore, and Hyderabad for this condition.

MODULE 23 — EXISTS / NOT EXISTS

What is EXISTS?

EXISTS checks whether a related record exists.

It returns conceptually:

TRUE / FALSE

Why use it?

We want:

Give me students who have at least one enrollment.

```
SELECT
    s.student_id,
    s.student_name
FROM students s
WHERE EXISTS (
    SELECT 1
    FROM enrollments e
    WHERE e.student_id = s.student_id
);
```

PostgreSQL checks each student:

Arun

```
↓
```

Enrollment exists?

YES

```
↓
Show Arun
```

NOT EXISTS does the opposite:

```
SELECT
    s.student_id,
    s.student_name
FROM students s
WHERE NOT EXISTS (
    SELECT 1
    FROM enrollments e
    WHERE e.student_id = s.student_id
);
```

Result in our project:

Ramesh

Deepa

The SELECT 1 is used because we only care whether a matching row exists; we don't need to return columns from the subquery.

MODULE 24 — ANY

What is ANY?

ANY means the comparison needs to be true for at least one value returned by the subquery.

Our partial payments are:

20000

10000

```
15000
```

Query:

```
SELECT
    course_name,
    course_fee
FROM courses
WHERE course_fee > ANY (
    SELECT amount_paid
    FROM payments
    WHERE payment_status = 'Partial'
);
```

Suppose course fee = 20000.

PostgreSQL checks:

```
20000 > 20000 → FALSE
20000 > 10000 → TRUE
20000 > 15000 → TRUE
At least one is TRUE.
```

Therefore:

```
TRUE
```

Easy memory:

ANY = at least one

MODULE 25 — ALL

What is ALL?

ALL means the comparison must be true for every value returned by the subquery.

Using the same partial payments:

20000

10000

```
15000
```

Query:

```
SELECT
    course_name,
    course_fee
FROM courses
WHERE course_fee > ALL (
    SELECT amount_paid
    FROM payments
    WHERE payment_status = 'Partial'
);
```

For ₹25,000:

```
25000 > 20000 → TRUE
25000 > 10000 → TRUE
25000 > 15000 → TRUE
ALL TRUE
→ Return course
```

For ₹20,000:

```
20000 > 20000 → FALSE
20000 > 10000 → TRUE
20000 > 15000 → TRUE
One FALSE
→ Don't return course
```

So remember:

```
ANY → At least one must satisfy
ALL → Every one must satisfy
```

This is exactly how the ANY/ALL examples in your project are structured.

MODULE 26 — CASE

What is CASE?

```
CASE adds IF/ELSE-style business logic inside SQL.
```

Why do we use it?

Database values may be numeric, but business users often need meaningful categories.

For example, course fee:

```
15000
20000
25000
35000
```

Management might want:

Low Cost

Medium Cost

Premium

We can create that dynamically:

```
SELECT
    course_name,
    course_fee,
    CASE
        WHEN course_fee < 20000
            THEN 'Low Cost'
        WHEN course_fee BETWEEN 20000 AND 30000
            THEN 'Medium Cost'
        ELSE 'Premium'
    END AS fee_category
FROM courses
ORDER BY course_fee;
```

How PostgreSQL checks it:

course_fee = 15000

Is < 20000?

YES

```
↓
Low Cost
```

For:

course_fee = 25000

Is < 20000?

NO

Is between 20000 and 30000?

YES

```
↓
Medium Cost
```

For:

course_fee = 35000

```
First condition → NO
Second condition → NO
ELSE
↓
Premium
Your project uses exactly these three fee categories.
The most important overall understanding
```

Don't learn these as 26 unrelated commands. Think about one business question:

Management wants to know how many Chennai students enrolled in each course, but show only courses having at least 2 Chennai students, highest count first.

That starts combining everything:

```
SELECT
   c.course_name,
```

COUNT(*) AS total_students

```
FROM students s
INNER JOIN enrollments e
   ON s.student_id = e.student_id
INNER JOIN courses c
   ON e.course_id = c.course_id
WHERE s.city = 'Chennai'
GROUP BY c.course_id, c.course_name
HAVING COUNT(*) >= 2
ORDER BY total_students DESC;
```

Now read it in simple English:

```
FROM students
→ Start with students
JOIN enrollments
→ Find what they enrolled in
JOIN courses
→ Get the course name
WHERE city = Chennai
→ Keep only Chennai students
GROUP BY course
→ Put students course-wise
COUNT(*)
→ Count students in each course
HAVING COUNT(*) >= 2
→ Keep courses having at least 2 students
ORDER BY total_students DESC
→ Highest student count first
```

That is the main goal of learning SQL. First understand the business question, then decide which SQL commands are needed to answer it.

Quick Revision — When to Use Each SQL Feature

Feature	Simple Meaning	Institute Example
SELECT	Read data	Show students
DISTINCT	Remove duplicates	Unique cities
WHERE	Filter rows	Only Chennai students
ORDER BY	Sort	Highest fee first
LIMIT/OFFSET	Pagination	5 students per page
MIN/MAX	Smallest/largest	Lowest/highest fee
COUNT	Count rows	Total students
SUM	Add values	Total collection
AVG	Average	Average fee
LIKE/ILIKE	Pattern search	Names starting A
IN	Match list	Selected cities
BETWEEN	Range	Fee 20k–30k
ALIAS	Temporary name	student_name AS name
INNER JOIN	Only matches	Enrolled students
LEFT JOIN	All left + matches	Include non-enrolled
RIGHT JOIN	All right + matches	Courses with no students
FULL JOIN	Both sides	Matched/unmatched
CROSS JOIN	All combinations	Student × Course
UNION	Combine rows	Student + trainer names
GROUP BY	Create groups	Students per city
HAVING	Filter groups	Cities >1 student
EXISTS	Check related row	Has enrollment
ANY	At least one	Fee > any partial payment
ALL	Every one	Fee > all partial payments
CASE	IF/ELSE logic	Fee category